

SDD — Módulo 4: Tablas Hash

1. Información general del proyecto

Nombre del proyecto: Visualizador Web de Estructuras de Datos

Módulo: Tablas Hash

Versión: 0.1.0

Estado: Especificación inicial para implementación del cuarto módulo

Tecnologías: Python, Flask, HTML5, CSS3, JavaScript, Jinja2

Tipo de proyecto: Aplicación web didáctica

Público objetivo: Estudiantes, docentes y monitores de estructuras de datos

Descripción general

El módulo de tablas hash permitirá a los estudiantes comprender el funcionamiento de una tabla hash implementada mediante **encadenamiento separado**, donde cada bucket reutiliza una lista enlazada.

El módulo permitirá insertar, buscar, actualizar, eliminar y visualizar pares clave-valor, además de explicar conceptos como función hash, índice calculado, buckets, colisiones, factor de carga y redimensionamiento.

Este módulo debe integrarse con la arquitectura general del proyecto y conservar las mismas especificaciones de calidad, documentación, separación de capas, ayudas didácticas, trazabilidad, pruebas y criterios de aceptación definidos para los módulos anteriores.

Objetivo del módulo

Diseñar e implementar un módulo web didáctico que permita interactuar con una tabla hash, visualizar sus buckets, observar colisiones y comprender el efecto del factor de carga y del redimensionamiento automático.

Alcance inicial

La primera versión del módulo debe incluir:

- Tabla hash genérica.
- Inserción de pares clave-valor.
- Actualización de valor cuando la clave ya existe.
- Búsqueda por clave.
- Verificación de existencia de clave.
- Eliminación por clave.
- Consulta de claves.
- Consulta de valores.
- Consulta de pares clave-valor.
- Consulta de tamaño.
- Consulta de capacidad.
- Consulta de factor de carga.
- Limpieza de la tabla.
- Visualización de buckets.

- Visualización de colisiones mediante listas enlazadas.
- Visualización didáctica del índice hash calculado.
- Visualización del redimensionamiento automático.

Fuera de alcance para esta fase

- Hashing abierto con sondeo lineal, cuadrático o doble hashing.
 - Funciones hash personalizadas desde la interfaz.
 - Persistencia en base de datos.
 - Autenticación de usuarios.
 - Evaluación automática de estudiantes.
 - Comparación de rendimiento entre diferentes estrategias de hashing.
 - Implementaciones en C, aunque la arquitectura debe quedar preparada para ellas.
-

2. Visión del sistema

Problema que resuelve

Las tablas hash combinan varios conceptos que pueden resultar abstractos para estudiantes: claves, valores, función hash, índice, buckets, colisiones, listas enlazadas, factor de carga y redimensionamiento. El módulo busca hacer visibles estos conceptos mediante una representación gráfica e interactiva.

Usuarios objetivo

- Estudiantes de estructuras de datos.
- Estudiantes de algoritmos.
- Docentes de programación.
- Monitores académicos.
- Desarrolladores educativos.

Beneficios esperados

- Comprensión visual de la relación clave-valor.
- Comprensión del cálculo de índice mediante hash.
- Identificación de colisiones.
- Visualización de encadenamiento separado.
- Comprensión del factor de carga.
- Visualización del redimensionamiento automático.
- Base extensible para otras técnicas de hashing.
- Preparación para futuras implementaciones en C.

Contexto de uso

El módulo se usará desde un navegador web dentro de la misma aplicación Flask del proyecto principal.

3. Requisitos funcionales

ID	Requisito	Descripción	Prioridad	Actor
RF-H-001	Acceso al módulo	Permitir ingresar al módulo de tablas hash desde la página principal	Alta	Estudiante
RF-H-002	Crear tabla hash	Permitir crear una tabla hash con capacidad inicial	Alta	Estudiante
RF-H-003	Validar capacidad	Validar que la capacidad inicial sea positiva	Alta	Estudiante
RF-H-004	Insertar par clave-valor	Permitir insertar una clave y un valor	Alta	Estudiante
RF-H-005	Actualizar valor	Si la clave ya existe, actualizar su valor	Alta	Estudiante
RF-H-006	Buscar clave	Permitir buscar una clave y retornar su valor o indicar que no existe	Alta	Estudiante
RF-H-007	Verificar existencia	Permitir verificar si una clave existe	Media	Estudiante
RF-H-008	Eliminar clave	Permitir eliminar una clave si existe	Alta	Estudiante
RF-H-009	Consultar claves	Mostrar todas las claves almacenadas	Media	Estudiante
RF-H-010	Consultar valores	Mostrar todos los valores almacenados	Media	Estudiante
RF-H-011	Consultar items	Mostrar todos los pares clave-valor	Media	Estudiante
RF-H-012	Consultar factor de carga	Mostrar el factor de carga actual	Alta	Estudiante
RF-H-013	Consultar tamaño	Mostrar cantidad de entradas almacenadas	Media	Estudiante
RF-H-014	Consultar capacidad	Mostrar capacidad actual de la tabla	Media	Estudiante
RF-H-015	Limpiar tabla	Permitir eliminar todas las entradas	Media	Estudiante
RF-H-016	Visualizar buckets	Mostrar cada bucket de la tabla	Alta	Estudiante
RF-H-017	Visualizar colisiones	Mostrar colisiones como cadenas dentro del bucket correspondiente	Alta	Estudiante

ID	Requisito	Descripción	Prioridad	Actor
RF-H-018	Visualizar índice hash	Mostrar el índice calculado para una clave durante insertar, buscar o eliminar	Alta	Estudiante
RF-H-019	Visualizar redimensionamiento	Indicar cuando el factor de carga supera el umbral y la tabla cambia de capacidad	Alta	Estudiante
RF-H-020	Ayuda del módulo	Proveer ayuda general sobre tablas hash	Alta	Estudiante
RF-H-021	Ayuda por operación	Proveer ayuda para insertar, buscar, eliminar, colisiones y redimensionamiento	Alta	Estudiante
RF-H-022	Manejo de errores	Traducir excepciones técnicas a mensajes didácticos	Alta	Estudiante
RF-H-023	Estado de sesión	Mantener temporalmente el estado de la tabla activa	Media	Estudiante
RF-H-024	Separación lógica/visual	Separar TAD, adaptadores, servicios, rutas, plantillas y JavaScript	Alta	Desarrollador
RF-H-025	Preparación para C	Mantener contratos de adaptadores compatibles con futuras implementaciones en C	Alta	Desarrollador
RF-H-026	Plantillas reutilizables	Usar el mismo header, footer, navbar y layout base del proyecto	Alta	Desarrollador

4. Requisitos no funcionales

ID	Tipo	Descripción
RNF-H-001	Documentación	Todo el código debe estar documentado según estándares del lenguaje
RNF-H-002	Python	El código Python debe cumplir PEP8, usar docstrings y type hints cuando sea razonable
RNF-H-003	JavaScript	Separar eventos, renderizado, animaciones y comunicación con backend
RNF-H-004	HTML	Usar HTML semántico y plantillas Jinja2 reutilizables
RNF-H-005	CSS	Separar estilos base, layout, componentes y visualización de tabla hash

ID	Tipo	Descripción
RNF-H-006	Modularidad	La tabla hash debe estar encapsulada mediante adaptador
RNF-H-007	Mantenibilidad	Debe ser sencillo agregar nuevas estrategias de hashing
RNF-H-008	Seguridad	Validar entradas y evitar ejecución de código arbitrario
RNF-H-009	Usabilidad	La interfaz debe ser clara para estudiantes principiantes
RNF-H-010	Rendimiento	El sistema debe funcionar con tablas pequeñas y medianas de uso didáctico
RNF-H-011	Extensibilidad	La integración futura con C no debe afectar la interfaz web
RNF-H-012	Trazabilidad	Cada requisito debe mapearse con casos de uso, módulos y pruebas
RNF-H-013	Accesibilidad	Usar colores, etiquetas y textos alternativos para reforzar comprensión
RNF-H-014	Consistencia visual	Mantener el mismo sistema visual de los módulos anteriores
RNF-H-015	Claridad conceptual	La UI debe explicar colisiones, buckets y factor de carga en lenguaje didáctico

5. Casos de uso

CU-H-001 — Acceder al módulo de tablas hash

Actor: Estudiante

Descripción: El usuario ingresa al módulo de tablas hash.

Precondiciones: La aplicación principal está disponible.

Flujo principal:

1. El usuario abre la página principal.
2. Selecciona "Tablas Hash".
3. El sistema muestra la página principal del módulo.
4. El sistema muestra opciones para crear una tabla hash temporal.

Postcondición: El usuario puede crear o usar una tabla hash.

CU-H-002 — Crear tabla hash

Actor: Estudiante

Descripción: El usuario crea una tabla hash con capacidad inicial.

Precondiciones: El usuario está en el módulo.

Flujo principal:

1. El usuario ingresa una capacidad inicial.
2. El sistema valida que sea un entero positivo.
3. El adaptador crea la instancia de `TablaHash`.
4. El sistema muestra buckets vacíos.

Flujo alternativo: Si la capacidad no es positiva, se muestra mensaje didáctico.

Postcondición: Existe una tabla hash activa.

CU-H-003 — Insertar par clave-valor

Actor: Estudiante

Descripción: El usuario inserta una clave y un valor.

Precondiciones: Existe una tabla hash activa.

Flujo principal:

1. El usuario ingresa clave y valor.
2. El sistema valida los datos.
3. El sistema calcula o informa el índice hash asociado.
4. El adaptador invoca `insertar(clave, valor)`.
5. El sistema actualiza la visualización del bucket.
6. Si la clave ya existía, se informa que el valor fue actualizado.
7. Si ocurre redimensionamiento, se muestra explicación didáctica.

Postcondición: La tabla contiene la clave con el valor indicado.

CU-H-004 — Buscar clave

Actor: Estudiante

Descripción: El usuario busca una clave en la tabla hash.

Precondiciones: Existe una tabla hash activa.

Flujo principal:

1. El usuario ingresa una clave.
2. El sistema calcula o informa el bucket asociado.
3. El adaptador invoca `buscar(clave)`.
4. El sistema muestra el valor si existe.
5. El sistema resalta el bucket revisado.

Flujo alternativo: Si la clave no existe, se muestra mensaje didáctico.

Postcondición: El usuario conoce si la clave existe y cuál es su valor.

CU-H-005 — Verificar existencia de clave

Actor: Estudiante

Descripción: El usuario verifica si una clave está almacenada.

Precondiciones: Existe una tabla hash activa.

Flujo principal:

1. El usuario ingresa una clave.
2. El adaptador invoca `contiene(clave)`.
3. El sistema muestra si la clave existe.

Postcondición: El usuario recibe un resultado booleano explicado.

CU-H-006 — Eliminar clave

Actor: Estudiante

Descripción: El usuario elimina una clave.

Precondiciones: Existe una tabla hash activa.

Flujo principal:

1. El usuario ingresa la clave a eliminar.
2. El sistema valida la entrada.
3. El adaptador invoca `eliminar(clave)`.
4. El sistema actualiza el bucket correspondiente.
5. El sistema muestra si la eliminación fue exitosa.

Flujo alternativo: Si la clave no existe, se muestra mensaje didáctico.

Postcondición: La clave queda eliminada si existía.

CU-H-007 — Consultar contenido de la tabla

Actor: Estudiante

Descripción: El usuario consulta claves, valores o pares clave-valor.

Precondiciones: Existe una tabla hash activa.

Flujo principal:

1. El usuario selecciona consultar claves, valores o items.
2. El adaptador invoca `claves()`, `valores()` o `items()`.
3. El sistema muestra el resultado.

Postcondición: El usuario visualiza el contenido de la tabla.

CU-H-008 — Consultar métricas

Actor: Estudiante

Descripción: El usuario consulta tamaño, capacidad o factor de carga.

Precondiciones: Existe una tabla hash activa.

Flujo principal:

1. El usuario selecciona una métrica.
2. El sistema invoca el método correspondiente.
3. El sistema muestra el resultado y su significado.

Postcondición: El usuario comprende el estado de ocupación de la tabla.

CU-H-009 — Limpiar tabla

Actor: Estudiante

Descripción: El usuario elimina todos los pares almacenados.

Precondiciones: Existe una tabla hash activa.

Flujo principal:

1. El usuario selecciona limpiar.
2. El adaptador invoca `limpiar()`.
3. El sistema muestra buckets vacíos.

Postcondición: La tabla queda vacía, conservando su capacidad actual.

CU-H-010 — Consultar ayuda

Actor: Estudiante

Descripción: El usuario consulta ayuda del módulo o de una operación.

Precondiciones: El usuario está en la aplicación.

Flujo principal:

1. El usuario selecciona ayuda.
2. El sistema muestra definición conceptual.
3. El sistema muestra operaciones principales.
4. El sistema muestra complejidad, ejemplos y errores comunes.

Postcondición: El usuario obtiene apoyo didáctico.

6. Modelo de datos

6.1 Diccionario de datos

Entidad: HashTableSession

Atributo	Tipo	Descripción
session_id	string	Identificador de sesión
module	string	Valor fijo: hash_tables

Atributo	Tipo	Descripción
capacity	int	Capacidad actual de la tabla
size	int	Cantidad actual de entradas
load_factor	float	Factor de carga actual
state	dict/list	Estado serializable de la tabla
last_operation	string	Última operación ejecutada
last_result	dict/string	Resultado de la última operación
errors	list	Errores recientes

Entidad: HashTableVisualState

Atributo	Tipo	Descripción
structure	string	Valor fijo: hash_table
capacity	int	Capacidad actual
size	int	Cantidad de entradas
load_factor	float	Factor de carga
threshold	float	Umbral de redimensionamiento, inicialmente 0.75
buckets	list	Lista de buckets visuales
highlighted_bucket	int/null	Bucket resaltado por la última operación
resized	bool	Indica si hubo redimensionamiento
last_operation	dict	Resultado de la última operación

Entidad: HashBucketVisualState

Atributo	Tipo	Descripción
index	int	Índice del bucket
entries	list	Entradas almacenadas en ese bucket
collision_count	int	Cantidad de colisiones visibles
highlighted	bool	Indica si el bucket está resaltado

Entidad: HashEntryVisualState

Atributo	Tipo	Descripción
key	string	Clave visible
value	string	Valor visible

Atributo	Tipo	Descripción
position	int	Posición dentro del bucket
highlighted	bool	Indica si la entrada fue afectada por la última operación

7. Arquitectura del sistema

7.1 Estilo arquitectónico

El módulo de tablas hash debe seguir la misma arquitectura por capas del proyecto:

```

Usuario
  ↓
Frontend HTML/CSS/JavaScript
  ↓
Rutas Flask / Controladores
  ↓
Servicios de aplicación
  ↓
Adaptador de tabla hash
  ↓
TAD Python actual
  ↓
Futura implementación en C

```

7.2 Capas principales

Capa de presentación

Responsable de:

- Renderizar plantillas del módulo de tablas hash.
- Dibujar buckets y cadenas de colisión.
- Mostrar claves, valores e índices.
- Mostrar factor de carga y capacidad.
- Resaltar bucket calculado por hash.
- Mostrar ayuda y mensajes didácticos.

Capa de rutas Flask

Responsable de:

- Recibir solicitudes HTTP.
- Validar datos básicos.
- Invocar servicios.
- Retornar HTML o JSON.

No debe contener lógica interna de la tabla hash.

Capa de servicios

Responsable de:

- Coordinar operaciones.
- Manejar estado de sesión.
- Capturar excepciones.
- Traducir respuestas a mensajes didácticos.
- Entregar estados serializables al frontend.

Capa de adaptadores

Responsable de normalizar el acceso al TAD del módulo.

Adaptador previsto:

- HashTableAdapter.

Capa de dominio

Contiene el TAD original:

- `tabla_hash.py`.

Dependencias reutilizadas:

- `lista_enlazada.py`.

Esta capa debe permanecer independiente de Flask, plantillas y JavaScript.

8. Estructura de carpetas propuesta

```
visualizador_estructuras/  
|  
├─ app/  
|   ├─ routes/  
|   |   ├─ hash_table_routes.py  
|   |   └─ help_routes.py  
|   └─ services/  
|       ├─ hash_table_service.py  
|       ├─ session_service.py  
|       └─ help_service.py  
|   └─ adapters/  
|       ├─ base_adapter.py  
|       └─ hash_table_adapter.py  
|   └─ domain/
```

```

├── hash_tables/
│   ├── __init__.py
│   └── tabla_hash.py
├── sequential/
│   └── lista_enlazada.py
├── templates/
│   ├── base.html
│   ├── partials/
│   │   ├── header.html
│   │   ├── footer.html
│   │   ├── navbar.html
│   │   ├── messages.html
│   │   └── help_panel.html
│   ├── pages/
│   │   ├── hash_tables.html
│   │   └── hash_table_view.html
│   └── help/
│       ├── hash_tables_help.html
│       ├── hash_insert_help.html
│       ├── hash_search_help.html
│       ├── hash_delete_help.html
│       ├── hash_collision_help.html
│       └── hash_resize_help.html
├── static/
│   ├── css/
│   │   ├── hash_tables.css
│   │   └── hash_table_visualizer.css
│   ├── js/
│   │   ├── hash_table_api_client.js
│   │   ├── hash_table_visualizer.js
│   │   ├── hash_table_animations.js
│   │   └── hash_table_helpers.js
│   └── img/
├── tests/
│   ├── test_hash_table_adapter.py
│   ├── test_hash_table_routes.py
│   ├── test_hash_table_visual_state.py
│   ├── test_hash_table_operations.py
│   └── test_hash_table_help.py
├── docs/
│   └── hash_tables_sdd.md

```

└─ hash_tables_user_guide.md
└─ future_c_integration.md

9. Contratos de interfaz UI/UX

Pantalla del módulo de tablas hash

Debe incluir:

- Título: "Tablas Hash".
- Descripción general.
- Campo para capacidad inicial.
- Botón "Crear tabla".
- Botón "Ayuda del módulo".
- Explicación breve de buckets, colisiones y factor de carga.

Pantalla de visualización de tabla hash

Header / Navbar	
Nombre del módulo	
Capacidad, tamaño y factor de carga	
Panel de operaciones clave-valor	Visualización de buckets
Resultado de operación	
Explicación de hash, colisión o resize	
Errores didácticos	
Footer	

Contrato visual para tabla hash

```
{  
  "structure": "hash_table",  
  "capacity": 17,  
  "size": 3,  
  "load_factor": 0.176,  
  "threshold": 0.75,  
  "highlighted_bucket": 4,  
  "resized": false,  
  "buckets": [  
    {  
      "index": 0,
```

```

    "entries": [],
    "collision_count": 0,
    "highlighted": false
  },
  {
    "index": 4,
    "entries": [
      {"key": "A", "value": "10", "position": 0, "highlighted": true}
    ],
    "collision_count": 0,
    "highlighted": true
  }
],
"last_operation": {
  "name": "insertar",
  "status": "success",
  "message": "La clave A fue insertada en el bucket 4."
}
}

```

10. Reglas de negocio

ID	Regla	Descripción
RB-H-001	Capacidad positiva	La tabla solo puede crearse con capacidad inicial mayor que cero
RB-H-002	Clave requerida	Insertar, buscar, contiene y eliminar requieren clave
RB-H-003	Valor requerido	Insertar requiere clave y valor
RB-H-004	Actualización por clave	Si la clave ya existe, insertar debe actualizar el valor
RB-H-005	Eliminación segura	Eliminar retorna falso si la clave no existe y debe explicarse didácticamente
RB-H-006	Búsqueda sin error	Buscar retorna <code>None</code> si la clave no existe
RB-H-007	Factor de carga	El factor de carga debe mostrarse después de operaciones de modificación
RB-H-008	Redimensionamiento	Si el factor de carga supera 0.75, la tabla redimensiona su capacidad
RB-H-009	Estado serializable	El backend debe entregar estados JSON serializables

ID	Regla	Descripción
RB-H-010	Dominio independiente	El TAD no debe importar Flask ni depender de la interfaz
RB-H-011	Contrato estable	Los nombres internos de operaciones deben mantenerse estables
RB-H-012	Integración futura con C	El cambio a C debe ocurrir detrás del adaptador
RB-H-013	No inventar operaciones	Si una operación no existe en el TAD, debe marcarse como pendiente
RB-H-014	Validación de entradas	Toda clave, valor y capacidad deben validarse antes de ejecutar operaciones
RB-H-015	Visualización de colisiones	Si un bucket tiene más de una entrada, debe mostrarse como colisión por encadenamiento

11. Operaciones iniciales por TAD

TablaHash

Operaciones esperadas:

- `insertar(clave, valor)`.
- `buscar(clave)`.
- `contiene(clave)`.
- `eliminar(clave)`.
- `claves()`.
- `valores()`.
- `items()`.
- `factor_carga()`.
- `tamano()`.
- `capacidad()`.
- `limpiar()`.

Observaciones didácticas

- La tabla usa encadenamiento separado.
- Cada bucket reutiliza una lista enlazada.
- La función `_indice(clave)` calcula `hash(clave) % capacidad`.
- `_indice` y `_redimensionar` son métodos internos, por lo que la UI no debe depender directamente de ellos salvo decisión técnica documentada en el adaptador.
- El redimensionamiento ocurre cuando `factor_carga() > 0.75`.
- La nueva capacidad usada por el TAD es `capacidad_actual * 2 + 1`.

12. Especificación de algoritmos

Algoritmo general para ejecutar una operación de tabla hash

Entrada:

- Operación solicitada.
- Parámetros del usuario.
- Estado actual de sesión.

Proceso:

1. Validar que exista una tabla hash activa.
2. Validar que la operación pertenezca al conjunto soportado.
3. Validar parámetros de entrada.
4. Recuperar instancia desde sesión.
5. Invocar el adaptador correspondiente.
6. Ejecutar el método real del TAD.
7. Capturar excepciones.
8. Traducir resultado a mensaje didáctico.
9. Generar estado visual serializable.
10. Retornar respuesta HTML o JSON a la UI.

Salida:

- Estado actualizado.
- Resultado de operación.
- Mensaje didáctico.
- Datos para visualización.

Algoritmo didáctico para insertar

Entrada: Clave y valor.

Proceso:

1. Validar clave y valor.
2. Registrar capacidad previa.
3. Calcular o informar el bucket asociado.
4. Verificar si la clave ya existe.
5. Ejecutar `insertar(clave, valor)`.
6. Comparar capacidad previa contra capacidad posterior.
7. Si cambió la capacidad, informar redimensionamiento.
8. Actualizar visualización.

Salida: Tabla actualizada, bucket resaltado y mensaje didáctico.

Algoritmo didáctico para buscar

Entrada: Clave.

Proceso:

1. Validar clave.
2. Calcular o informar bucket asociado.
3. Ejecutar `buscar(clave)`.
4. Resaltar bucket revisado.
5. Mostrar valor o indicar que la clave no existe.

Salida: Resultado de búsqueda y visualización resaltada.

Algoritmo didáctico para eliminar

Entrada: Clave.

Proceso:

1. Validar clave.
2. Calcular o informar bucket asociado.
3. Ejecutar `eliminar(clave)`.
4. Mostrar si la clave fue eliminada.
5. Actualizar visualización.

Salida: Tabla actualizada y mensaje didáctico.

Algoritmo para renderizar tabla hash

Entrada: Estado serializado de la tabla hash.

Proceso:

1. Limpiar contenedor visual.
2. Crear fila o tarjeta por cada bucket.
3. Mostrar índice de bucket.
4. Mostrar entradas clave-valor dentro de cada bucket.
5. Si hay más de una entrada, marcar colisión.
6. Resaltar bucket de la última operación.
7. Mostrar capacidad, tamaño y factor de carga.
8. Mostrar indicador de redimensionamiento si aplica.

Salida: Tabla hash dibujada en pantalla.

13. API / servicios

Rutas web sugeridas

Endpoint	Método	Descripción
<code>/tablas-hash</code>	GET	Página del módulo de tablas hash
<code>/tablas-hash/visualizador</code>	GET	Página de visualización de tabla hash

Endpoint	Método	Descripción
<code>/api/tablas-hash/create</code>	POST	Crear tabla hash con capacidad inicial
<code>/api/tablas-hash/operation</code>	POST	Ejecutar operación sobre la tabla
<code>/api/tablas-hash/state</code>	GET	Obtener estado actual serializado
<code>/api/tablas-hash/reset</code>	POST	Reiniciar tabla hash
<code>/ayuda/tablas-hash</code>	GET	Ayuda general del módulo
<code>/ayuda/tablas-hash/<topic_id></code>	GET	Ayuda específica por operación o concepto

Servicio: HashTableService

Responsabilidades:

- Crear tabla hash.
- Ejecutar operaciones básicas.
- Consultar estado.
- Reiniciar tabla.
- Generar mensajes didácticos.

Métodos conceptuales:

```
create_hash_table(capacity=17)
execute_operation(operation, payload)
get_hash_table_state()
reset_hash_table()
```

Servicio: SessionService

Responsabilidades:

- Guardar estado temporal.
- Recuperar tabla activa.
- Limpiar sesión.
- Serializar y deserializar datos.

Servicio: HelpService

Responsabilidades:

- Obtener ayuda del módulo.
- Obtener ayuda por operación.
- Obtener ayuda por concepto: bucket, colisión, factor de carga, redimensionamiento.

Contrato de adaptador base

Cada adaptador debe cumplir una interfaz común:

```

create()
execute(operation_name, payload)
to_visual_state()
reset()
get_supported_operations()

```

Contrato de HashTableAdapter

```

create(capacity=17)
insert(key, value)
search(key)
contains(key)
delete(key)
keys()
values()
items()
load_factor()
size()
capacity()
clear()
to_visual_state()
reset()
get_supported_operations()

```

14. Trazabilidad

RF	Caso de uso	Módulo	Prueba
RF-H-001	CU-H-001	HashTable Routes	test_hash_table_module_loads
RF-H-002	CU-H-002	HashTableService	test_create_hash_table
RF-H-003	CU-H-002	HashTableAdapter	test_invalid_capacity
RF-H-004	CU-H-003	HashTableAdapter	test_insert_key_value
RF-H-005	CU-H-003	HashTableAdapter	test_update_existing_key
RF-H-006	CU-H-004	HashTableAdapter	test_search_key
RF-H-007	CU-H-005	HashTableAdapter	test_contains_key

RF	Caso de uso	Módulo	Prueba
RF-H-008	CU-H-006	HashTableAdapter	test_delete_key
RF-H-009	CU-H-007	HashTableAdapter	test_keys
RF-H-010	CU-H-007	HashTableAdapter	test_values
RF-H-011	CU-H-007	HashTableAdapter	test_items
RF-H-012	CU-H-008	HashTableAdapter	test_load_factor
RF-H-013	CU-H-008	HashTableAdapter	test_size
RF-H-014	CU-H-008	HashTableAdapter	test_capacity
RF-H-015	CU-H-009	HashTableAdapter	test_clear
RF-H-016	CU-H-003 a CU-H-009	HashTable Visualizer JS	test_bucket_visual_state
RF-H-017	CU-H-003	HashTable Visualizer JS	test_collision_visualization
RF-H-018	CU-H-003 a CU-H-006	HashTableAdapter	test_highlighted_bucket
RF-H-019	CU-H-003	HashTableAdapter	test_resize_detection
RF-H-020	CU-H-010	HelpService	test_hash_table_help_available
RF-H-021	CU-H-010	HelpService	test_hash_topic_help_available
RF-H-022	CU-H-002 a CU-H-009	Services / Exceptions	test_hash_table_error_messages
RF-H-023	CU-H-002 a CU-H-009	SessionService	test_hash_table_session_persistence
RF-H-024	Todos	Domain / Services / Adapters	test_no_flask_dependency_in_hash_domain
RF-H-025	Futuro	Adapter Layer	test_hash_table_adapter_contract_stability
RF-H-026	Todos	Templates	test_base_template_reuse

15. Plan de pruebas

Pruebas unitarias

Deben cubrir:

- HashTableAdapter.
- HashTableService.
- Conversión a estado visual.
- Manejo de errores.
- Validaciones de entrada.
- Inserción, actualización, búsqueda y eliminación.
- Factor de carga y redimensionamiento.

Pruebas de integración

Deben validar:

- Rutas Flask del módulo.
- Creación de tabla hash.
- Inserción de pares clave-valor.
- Actualización de valor.
- Eliminación de claves.
- Respuesta JSON/HTML.
- Persistencia temporal en sesión.

Pruebas de interfaz

Deben validar:

- Carga de plantillas.
- Botones visibles.
- Visualización de buckets.
- Visualización de colisiones.
- Visualización de factor de carga.
- Visualización de redimensionamiento.
- Mensajes de ayuda.

Pruebas didácticas

Deben validar que:

- Cada operación tenga explicación.
 - El índice hash se explique cuando sea posible.
 - Las colisiones sean visibles.
 - El factor de carga tenga explicación.
 - El redimensionamiento se informe claramente.
 - Los errores sean comprensibles.
-

16. Criterios de aceptación

La primera versión del módulo de tablas hash será aceptada cuando:

1. La aplicación Flask mantenga la navegación general.
2. Exista una página del módulo de tablas hash.
3. Se pueda crear una tabla hash con capacidad positiva.
4. Se bloquee la creación con capacidad inválida y se explique el error.
5. Se puedan insertar pares clave-valor.
6. Se pueda actualizar el valor de una clave existente.
7. Se pueda buscar una clave.
8. Se pueda verificar si una clave existe.
9. Se pueda eliminar una clave.
10. Se puedan consultar claves, valores e items.
11. Se pueda consultar tamaño, capacidad y factor de carga.
12. Se pueda limpiar la tabla.
13. La visualización muestre buckets e índices.
14. La visualización muestre colisiones por encadenamiento separado.
15. Se resalte el bucket afectado por la última operación.
16. Se informe si ocurre redimensionamiento.
17. Los errores sean controlados y explicados de forma didáctica.
18. El código de dominio no importe Flask.
19. Las plantillas reutilicen header, footer y navbar del proyecto.
20. Exista ayuda didáctica del módulo y de cada operación/concepto.
21. El código Python cumpla PEP8 y tenga docstrings.
22. El frontend esté separado en archivos CSS y JavaScript.
23. Existan pruebas iniciales del módulo.
24. El proyecto quede preparado para futuras implementaciones de TAD en C.

17. Gestión del proyecto

Metodología

Desarrollo basado en especificaciones, con implementación incremental por operaciones y visualización didáctica.

Herramientas sugeridas

- Python 3.
- Flask.
- Pytest.
- HTML5.
- CSS3.
- JavaScript.
- Git.
- Agente IA de desarrollo.

Fases

Fase 1 — Análisis del TAD TablaHash

- Revisar `tabla_hash.py`.
- Extraer clase, métodos y excepciones reales.
- Identificar dependencia con `ListaEnlazada`.
- Identificar limitaciones de visualización.

Fase 2 — Base del módulo

- Crear rutas de tablas hash.
- Crear plantillas del módulo.
- Integrar navegación desde la página principal.

Fase 3 — Adaptador

- Crear HashTableAdapter.
- Normalizar operaciones.
- Crear estado visual serializable.
- Detectar bucket afectado y redimensionamiento.

Fase 4 — Operaciones básicas

- Crear tabla.
- Insertar clave-valor.
- Buscar clave.
- Verificar existencia.
- Eliminar clave.
- Consultar claves, valores e items.
- Consultar métricas.
- Limpiar tabla.

Fase 5 — Visualización

- Crear visualizador de buckets.
- Representar colisiones por encadenamiento.
- Mostrar factor de carga.
- Mostrar redimensionamiento.
- Resaltar bucket afectado.

Fase 6 — Ayuda didáctica

- Crear ayuda general del módulo.
- Crear ayuda para operaciones.
- Crear ayuda para colisiones, factor de carga y redimensionamiento.

Fase 7 — Pruebas y documentación

- Agregar pruebas unitarias.
- Agregar pruebas de rutas.
- Documentar arquitectura.
- Documentar uso del módulo.

- Documentar preparación para C.

18. Integración con agentes de IA

18.1 Roles

Agente	Responsabilidad
Agente arquitecto	Mantener arquitectura, trazabilidad y separación de capas
Agente backend	Implementar Flask, servicios, adaptadores, sesión y pruebas backend
Agente frontend	Implementar plantillas, CSS, JS y visualización de tabla hash
Agente documental	Crear ayudas didácticas, README, guías y docstrings
Agente QA	Crear pruebas unitarias, integración y validación de aceptación

18.2 Prompt base para agente desarrollador

Actúa como un agente desarrollador senior especializado en Python, Flask, HTML5, CSS3, JavaScript y arquitectura limpia para aplicaciones educativas.

Contexto del proyecto:

Debes construir un nuevo módulo de una aplicación web didáctica llamada "Visualizador Web de Estructuras de Datos". El objetivo del módulo es permitir que estudiantes aprendan tablas hash mediante visualizaciones interactivas.

La aplicación general tiene módulos de estructuras secuenciales, estructuras jerárquicas, grafos y ahora tablas hash.

En esta fase solo debes implementar el módulo de tablas hash.

TAD disponible inicialmente para este módulo:

- tabla_hash.py

Dependencia de dominio reutilizada por el TAD TablaHash:

- lista_enlazada.py

Objetivo técnico:

Crear un módulo Flask modular que permita crear una tabla hash, insertar, buscar, verificar, eliminar y visualizar pares clave-valor, mostrando buckets, colisiones, factor de carga y redimensionamiento. La lógica del TAD debe estar completamente separada de la capa visual.

Restricciones obligatorias:

1. No mezclar lógica de tabla hash con rutas Flask ni plantillas HTML.
2. No modificar el TAD original salvo que sea estrictamente necesario y documentado.

3. No inventar operaciones que no existan en el TAD. Si una operación requerida no existe, marcarla como pendiente.
4. Todo código Python debe cumplir PEP8.
5. Todo módulo, clase y función Python debe tener docstrings.
6. Usar nombres claros y type hints cuando sea razonable.
7. No usar eval.
8. Validar entradas de usuario.
9. Todos los errores deben traducirse a mensajes didácticos.
10. La interfaz debe usar plantillas reutilizables con header, footer, navbar y layout base.
11. El frontend debe estar separado en HTML, CSS y JavaScript.
12. La arquitectura debe quedar preparada para que en una fase futura los TAD puedan implementarse en C.
13. No implementar estrategias de hashing no presentes en el TAD.
14. No exponer directamente objetos internos no serializables al frontend.
15. No depender de métodos privados del TAD salvo decisión técnica documentada en el adaptador.

Arquitectura esperada:

Implementa una arquitectura por capas:

- Capa de presentación:
 - templates/
 - static/css/
 - static/js/
- Capa de rutas:
 - app/routes/hash_table_routes.py
 - app/routes/help_routes.py
- Capa de servicios:
 - app/services/hash_table_service.py
 - app/services/session_service.py
 - app/services/help_service.py
- Capa de adaptadores:
 - app/adapters/base_adapter.py
 - app/adapters/hash_table_adapter.py
- Capa de dominio:
 - app/domain/hash_tables/
 - Allí debe ubicarse tabla_hash.py.

Funcionalidades mínimas:

1. Página del módulo de tablas hash.
2. Navegación desde la página principal.
3. Crear tabla hash con capacidad inicial.
4. Insertar par clave-valor.
5. Actualizar valor cuando la clave ya existe.
6. Buscar clave.
7. Verificar existencia de clave.

8. Eliminar clave.
9. Consultar claves.
10. Consultar valores.
11. Consultar pares clave-valor.
12. Consultar tamaño.
13. Consultar capacidad.
14. Consultar factor de carga.
15. Limpiar tabla.
16. Visualizar buckets.
17. Visualizar colisiones por encadenamiento separado.
18. Resaltar bucket afectado.
19. Mostrar redimensionamiento si ocurre.
20. Mensajes didácticos después de cada operación.
21. Manejo de errores para capacidad inválida y entradas incompletas.
22. Sección de ayuda general para tablas hash.
23. Ayuda específica para operaciones y conceptos.

Contrato de adaptador:

Cada adaptador debe implementar una interfaz común equivalente a:

- create()
- execute(operation_name, payload)
- to_visual_state()
- reset()
- get_supported_operations()

El método to_visual_state() debe devolver datos serializables, por ejemplo:

```
{
  "structure": "hash_table",
  "capacity": 17,
  "size": 0,
  "load_factor": 0,
  "threshold": 0.75,
  "buckets": [],
  "highlighted_bucket": null,
  "last_operation": {
    "name": "create_hash_table",
    "status": "success",
    "message": "Tabla hash creada correctamente."
  }
}
```

Pruebas requeridas:

Crea pruebas para:

1. Carga del módulo de tablas hash.
2. Creación de tabla con capacidad válida.
3. Bloqueo de capacidad inválida.
4. Inserción de clave-valor.
5. Actualización de clave existente.
6. Búsqueda de clave existente e inexistente.

7. Verificación de existencia de clave.
8. Eliminación de clave existente e inexistente.
9. Consulta de claves, valores e items.
10. Consulta de tamaño, capacidad y factor de carga.
11. Limpieza de tabla.
12. Conversión a estado visual.
13. Visualización de colisiones.
14. Detección de redimensionamiento.
15. Persistencia temporal del estado en sesión.
16. Disponibilidad de ayuda didáctica.

Criterios de aceptación:

La tarea se considera completa si:

1. La aplicación Flask ejecuta sin errores.
2. El módulo de tablas hash está disponible desde la navegación.
3. Se puede crear una tabla hash con capacidad positiva.
4. Las operaciones disponibles se obtienen desde adaptadores.
5. El estado visual se actualiza después de cada operación.
6. La visualización muestra buckets, entradas y colisiones.
7. Se informa el factor de carga.
8. Se informa el redimensionamiento cuando ocurre.
9. El código de dominio no importa Flask.
10. Las plantillas reutilizan header, footer y navbar.
11. Existe ayuda didáctica del módulo y de cada concepto.
12. Existen pruebas iniciales.
13. El proyecto queda preparado para futura integración de TAD implementados en C.

Prohibiciones explícitas:

- No generar código monolítico.
- No poner lógica de tabla hash en las rutas Flask.
- No poner lógica de negocio en JavaScript si debe estar en backend.
- No usar eval.
- No ignorar excepciones.
- No inventar métodos inexistentes en el TAD.
- No eliminar archivos TAD originales.
- No acoplar la UI directamente a una clase concreta del dominio.
- No implementar técnicas de hashing no disponibles en el TAD.
- No depender de atributos privados sin documentarlo como decisión técnica temporal.

Formato de salida esperado:

1. Primero crea un plan de implementación detallado antes de desarrollar la aplicación.
2. El plan debe incluir:
 - análisis del TAD suministrado;
 - mapa entre operaciones de UI y métodos reales;
 - estructura final de carpetas;
 - fases de implementación;
 - componentes backend, frontend, servicios, adaptadores y pruebas;
 - riesgos técnicos;

- operaciones pendientes o no soportadas;
 - criterios de validación antes de codificar.
3. Después de presentar el plan, desarrolla la aplicación siguiendo ese plan.
 4. Muestra el árbol de carpetas creado.
 5. Explica brevemente la responsabilidad de cada carpeta.
 6. Implementa los archivos necesarios.
 7. Incluye comentarios y docstrings.
 8. Incluye pruebas.
 9. Incluye instrucciones para ejecutar:
 - creación de entorno virtual;
 - instalación de dependencias;
 - ejecución de Flask;
 - ejecución de pruebas.
 10. Indica claramente cualquier operación pendiente porque no exista en el TAD suministrado.

Antes de codificar:

Analiza el TAD suministrado y extrae sus clases, métodos y excepciones reales. Usa esa información para construir el adaptador. Si existe discrepancia entre esta especificación y el TAD real, respeta el TAD real y documenta la diferencia.

19. Control de versiones

Rama principal

main

Debe contener solo versiones estables.

Rama de desarrollo

develop

Para integración de funcionalidades.

Ramas sugeridas

feature/hash-tables-module
feature/hash-table-routes
feature/hash-table-adapter
feature/hash-table-visualizer
feature/hash-table-help
feature/hash-table-tests

Convención de commits

```
feat: agrega módulo de tablas hash
feat: agrega adaptador de tabla hash
feat: agrega visualizador de buckets
feat: agrega visualización de colisiones
fix: corrige validación de capacidad inválida
docs: agrega ayuda didáctica para factor de carga
test: agrega pruebas para tabla hash
refactor: separa visualización de operaciones hash
```

20. Decisiones arquitectónicas importantes

ID	Decisión	Justificación
DA-H-001	Separar dominio y presentación	Evita acoplamiento entre TAD y Flask
DA-H-002	Usar adaptador	Permite normalizar operaciones y preparar integración con C
DA-H-003	Serializar estado visual	Facilita comunicación backend/frontend
DA-H-004	Usar plantillas base	Mantiene consistencia con módulos anteriores
DA-H-005	Visualizar buckets	Hace visible la estructura interna de la tabla
DA-H-006	Visualizar colisiones	Refuerza el concepto de encadenamiento separado
DA-H-007	Mostrar factor de carga	Explica cuándo se aproxima un redimensionamiento
DA-H-008	Mantener TAD original	Reduce riesgo de alterar implementaciones existentes
DA-H-009	Marcar pendientes técnicos	Evita que agentes IA inventen operaciones o datos no disponibles
DA-H-010	Preparar integración con C	Permite cambiar implementación sin rediseñar la UI